

Manual del equipo — Cresco

Versión 1.2 · 8 de agosto de 2026 · Equipo Neofix · RevenueCat Shipaton 2026, categoría Next Gen · Envío del equipo: 28 de septiembre

Corrige y reemplaza la versión 1.0 del 3 de agosto de 2026. Ver el detalle de cambios en [docs/99-archivo/ERRATA-2026-08.md](#) . Esta versión 1.2 reinicia el calendario (la semana 1 empieza el 9 de agosto) y fija dónde vive el código: [servidor/](#) y [movil/](#) . Ver [Contexto/DECISIONS.md](#) , decisiones S1-S6.

Cómo usar este manual

Este es el documento maestro. Contiene todo el proyecto: qué construimos, quién hace qué, en qué fechas y cómo sabemos que una fase salió bien.

Cada persona tiene además **su propio manual de rol** con el detalle paso a paso. Léanlos en este orden: primero este completo, después el suyo.

La última sección es un **bloque de contexto para inteligencia artificial**. Si se traban y quieren preguntarle a una IA, peguen ese bloque antes de su pregunta y la respuesta llegará ajustada a este proyecto en vez de ser genérica.

Parte 1 — El proyecto

1.1 Qué es Cresco

Una aplicación Android de comunicación entre **docentes** y **representantes legales** de escuelas del Ecuador, centrada en el seguimiento de responsabilidad y conducta del estudiante.

El estudiante nunca es usuario de la aplicación. Solo es sujeto de datos.

Hay dos aplicaciones distintas dentro de una misma base de código:

- **App del docente:** herramienta de trabajo diaria. Crea el curso, registra acciones, toma asistencia, redacta el reporte del día.
- **App del representante:** informativa y simple. Abre y ve el reporte de hoy de su hijo, con opciones para reclamar una acción injusta o agendar una cita.

1.2 Qué problema resuelve

Hoy la comunicación escuela-familia vive en grupos de mensajería desordenados, donde lo importante se pierde. El docente no tiene tiempo de hablar con treinta familias y el representante se entera de la conducta de su hijo al final del parcial, cuando ya es tarde para ayudar.

1.3 El mecanismo central: el puntaje

Cada estudiante empieza cada parcial con 60 puntos. Sube y baja según su responsabilidad y su comportamiento.

Concepto	Valor
Puntaje base al iniciar el parcial	60
Piso	0
Techo	100
Acción positiva	+1 por defecto, el docente puede subirla a +2
Acción negativa de responsabilidad	siempre -1, fija
Acción negativa de disciplina	-1 por defecto, hasta -3
Tope de ganancia por día	+4
Tope de pérdida por día	-5

El puntaje nunca modifica calificaciones académicas. Es un apoyo para cuando el docente califique conducta al cierre del parcial.

Al terminar el parcial, el puntaje se congela y el estudiante vuelve a 60 en el siguiente.

Franjas de conducta

El puntaje se traduce a una frase orientadora para el representante:

Rango	Franja
0 – 15	Situación crítica
16 – 30	Muy por debajo
31 – 50	Por debajo de lo esperado
51 – 60	En el punto de partida
61 – 80	Buen desempeño
81 – 100	Excelente

1.4 Los cinco tipos de acción

Tipo	Alcance	Puntos	¿Bitácora?	¿Reclamable?
Positiva	Un estudiante	+1 a +2	Sí	No
Negativa	Un estudiante	-1 a -3	Sí	Sí
Nota	Un estudiante	0	No	No
Anuncio	Todo el curso	0	No	No
Evento	Todo el curso	0	No	No

Las tres primeras van a la tabla `accion_registrada`. Las dos últimas van a `comunicado_curso`, porque alcanzan a todo el curso y no a un estudiante.

El **evento** aparece como bloque expandible al final del reporte diario, desde hasta 7 días antes de su fecha y hasta el día siguiente.

1.5 El flujo completo

1. El docente se registra y crea su curso escribiendo el nombre de su escuela como texto libre.
Define el año lectivo y de 2 a 3 parciales.

2. El docente genera un **código de invitación al curso**, válido 30 días.
3. El representante se registra, canjea el código, **registra los datos de su hijo** y acepta el consentimiento de tratamiento de datos.
4. El estudiante queda **pendiente de aprobación**. El docente lo revisa, completa lo que falte y lo aprueba. Ahí nace la matrícula con 60 puntos.
5. Durante el día, el docente asigna acciones y toma asistencia.
6. Al terminar la clase, el docente llena el reporte general del curso —campos etiquetados, ninguno obligatorio— y lo publica.
7. Se genera el reporte de cada estudiante y se notifica a su representante.
8. El representante abre la app y ve el reporte del día. Si hay una acción negativa que le parece injusta, abre una inconformidad. Si quiere hablar, agenda una cita en un bloque de 15 minutos que el docente publicó.

1.6 Decisiones ya tomadas (no se rediscuten)

Cada una está registrada como ADR en `docs/01-arquitectura/adr/`.

#	Decisión	Motivo corto
ADR-001	PostgreSQL + Drizzle ORM	Modelo relacional normalizado; tipos derivados del esquema
ADR-002	Next.js (servidor) + Expo (apps)	Tauri quedó descartado: no tiene SDK de RevenueCat
ADR-003	BetterAuth con plugin de organización	La organización es la institución declarada por el docente
ADR-004	Permisos en dos capas: código + RLS	Un bug de la app no debe poder filtrar datos de un menor
ADR-005	El puntaje es derivado, nunca un contador	Se recalcula desde las acciones vigentes
ADR-006	RevenueCat es la fuente de verdad de pagos	Requisito obligatorio del Shipaton
ADR-007	Entrega sin publicación en tienda, con repositorio abierto	Next Gen evalúa por video y repositorio; reemplaza la consecuencia de ADR-002 sobre Play Console

Aviso importante sobre `shadcn/ui` : solo funciona en web, no en React Native. Para las apps se usa NativeWind con componentes propios. shadcn queda reservado para el panel web de la dirección en la

1.7 RevenueCat: el requisito que no se negocia

El Shipaton exige el SDK de RevenueCat integrado y funcionando. **Si esto falla, no importa qué tan buena esté la app.** Por eso:

- Es responsabilidad de **Persona C** y arranca en la **semana 1**, no al final.
- Debe estar probado en un **dispositivo Android real**, no solo en emulador.
- RevenueCat en Expo exige un *development build*; **no funciona en Expo Go**.
- La categoría **Next Gen del Shipaton no exige cuenta de Google Play Console ni publicación en tienda**: se evalúa por video de demostración (menos de 2 minutos) y repositorio de código abierto público. Sigue pendiente confirmar en la documentación oficial de RevenueCat qué modo de prueba de compras funciona sin una app publicada en tienda.

Planes

Código	Audiencia	Qué da
REP_FREE	Representante	2 reportes anteriores, con publicidad
REP_PREMIUM_MENSUAL	Representante	Sin publicidad, 7 reportes, PDF libre
REP_PREMIUM_BIMESTRAL	Representante	Igual, ciclo de 2 meses ("por parcial")
DOC_FREE	Docente	1 curso activo, 40 estudiantes
DOC_PRO	Docente	5 cursos activos, 60 estudiantes, plantillas propias

1.8 Licencia y repositorio público

Next Gen exige un repositorio **público y de código abierto**, con un archivo `LICENSE` en la raíz, antes del envío. El equipo eligió **AGPL-3.0** con posibilidad de licenciar en paralelo en términos comerciales a colegios o clientes que lo requieran, ya que Neofix es titular único del copyright. El archivo **ya está en la raíz**; el repositorio arranca privado y se hace público solo antes de enviar.

Consecuencia práctica para los tres: todo lo que commiteen va a ser público. Nunca suban claves, `.env`, keystore ni datos reales de estudiantes — ni siquiera "temporalmente", porque el historial de Git conserva lo que se borró después.

Parte 2 — Cómo trabajamos

2.1 Los tres roles

	Persona A — Núcleo	Persona B — Conducta	Persona C — Interacción e infra
En una frase	Quién es quién y a qué tiene derecho	El corazón funcional del producto	Lo que rodea y sostiene la app
Dueño de	Identidad, curso, estudiantes, vinculación, permisos	Acciones, puntaje, asistencia, reportes	Citas, alertas, RevenueCat, notificaciones, CI
Archivo de esquema	<code>db/schema/nucleo.ts</code>	<code>db/schema/conducta.ts</code>	<code>db/schema/interaccion.ts</code>
Pantallas	D1–D8, P1–P3	D9–D13, P4–P6	D14–D19, P7–P12
Riesgo principal	Bloquea a los otros dos en semana 1	Es quien más lógica escribe	Depende de trámites externos

2.2 La regla de oro

Nadie edita el módulo de otro.

Si B necesita un campo nuevo en `matricula`, no lo agrega: se lo pide a A en el grupo y A lo agrega en una migración. Esto evita el 90 % de los conflictos de fusión, que es lo que hunde a los equipos de tres.

Superficie compartida (cambiarla exige acuerdo de los tres):

- `db/schema/enums.ts`
- `api/openapi.yaml`

2.3 Cómo trabajamos en paralelo sin esperarnos

Tres piezas lo hacen posible:

1. `api/openapi.yaml` permite levantar un servidor falso:

```
npx @stoplight/prism-cli mock api/openapi.yaml
```

Quien construye una pantalla recibe datos válidos aunque el backend no exista.

2. `db/schema/` da los tipos de TypeScript compartidos: los tres nombran los mismos campos.
3. `db/schema/enums.ts` garantiza que los valores de estado coincidan entre servidor y app.

Honestidad sobre la semana 1: no es del todo paralela. Persona A debe dejar corriendo la base de datos y la autenticación antes de que los demás trabajen contra datos reales. Mientras tanto, B escribe el motor de puntaje con pruebas unitarias (no necesitan servidor) y C monta el repositorio, el CI y RevenueCat. Del día 4 en adelante sí es paralelo completo.

2.4 Ritmo semanal

- **Lunes, 20 minutos:** qué hizo cada uno, qué sigue, qué le bloquea.
- **Viernes, 10 minutos:** riesgos y avance contra el plan.
- **Regla del medio día:** si algo te bloquea más de medio día, lo dices ese mismo día. No el lunes. Lo que hunde un proyecto de seis semanas no es el atraso, es enterarse tarde.

2.5 Ramas y revisiones

```
main          <- protegida, nadie empuja directo
├─ develop    <- integración
│   └─ feat/asignar-accion-tope-diario
│   └─ fix/reporte-diario-204
│   └─ chore/configurar-ci
```

- Commits en formato convencional: `feat(acciones): valida tope diario de -5`
- **Nadie fusiona su propio PR.** Se necesita una aprobación.
- Un PR que toque datos de estudiantes exige las tres verificaciones de `docs/01-arquitectura/matriz-permisos.md`.

2.6 Definición de "terminado"

Una tarea está terminada cuando cumple las cinco:

1. El endpoint responde según `openapi.yaml`, incluidos sus códigos de error.
2. La pantalla maneja los tres estados: cargando, vacío y error.
3. Si toca datos de estudiantes, tiene una prueba de que un usuario sin vínculo recibe `403`.
4. Fue revisada y aprobada por otra persona del equipo.
5. Funciona en un dispositivo Android real, no solo en el emulador.

2.7 Nunca se sube al repositorio

Claves de RevenueCat · credenciales de base de datos · keystore de firma · archivos `.env` · **datos reales de estudiantes o representantes.**

El Excel que les pase un profesor para probar la carga **no entra al repositorio**. Se usan datos ficticios generados.

Recordatorio de Next Gen: el repositorio se hará **público** antes de enviar. Nunca subir claves, `.env`, keystore ni datos reales de estudiantes.

Parte 3 — Calendario y criterios de éxito

3.1 Fechas

Calendario reiniciado el 8 de agosto. El plan original (semana 1 desde el 4 de agosto) había quedado atrás — sus tareas no se habían ejecutado. Se reinicia el reloj desde una fecha real: mañana. Ver decisiones S1-S6 en [Contexto/DECISIONS.md](#).

Fase	Fechas	Foco
Día 0	hoy, sábado 8 de agosto	Leer manuales, instalar herramientas
Semana 1	9 – 15 de agosto	Cimientos
Semana 2	16 – 22 de agosto	Flujo del docente
Semana 3	23 – 29 de agosto	Flujo del representante
Semana 4	30 de agosto – 5 de septiembre	Punto de control
Semana 5	6 – 12 de septiembre	Funciones de apoyo
Semana 6	13 – 19 de septiembre	Cierre y demostración
Deadline interno	20 de septiembre	No es la fecha real de envío — quedan 8 días de margen
Envío del equipo	28 de septiembre	Fecha real de entrega — externa, no se movió con el reinicio del calendario
Cierre oficial Shipaton	30 de septiembre, 23:45 PDT	1 de octubre, 01:45 hora Ecuador — externa, no se movió

El colchón se comprimió: era de 13 días (15→28 sep.), ahora son **8 días** (20→28 sep.). Las tres semanas de pulido y prueba con un profesor real siguen existiendo pero con menos margen de error. Ver [Contexto/NEXT_STEPS.md](#), Paso 5. Siguen **sin ser** para ampliar alcance.

3.2 Semana 1 — Cimientos (9 al 15 de agosto)

Objetivo de la fase: que exista una base de datos con datos de prueba, que alguien pueda iniciar sesión y que RevenueCat responda en sandbox.

Rol	Tareas	Éxito de la fase
A	Postgres corriendo (E1/E2 ya aplicadas), BetterAuth funcionando dentro de <code>servidor/</code> , capa <code>db/acceso/</code> , políticas RLS, datos semilla	Un usuario se registra, inicia sesión y la consulta sin <code>app.user_id</code> devuelve cero filas
B	Catálogo de acciones semilla, función <code>recalcularPuntaje</code> en <code>servidor/lib/puntaje.ts</code> , pruebas de topes diarios	<code>npm test</code> pasa: registrar -3 y luego -3 el mismo día es rechazado por el tope de -5
C	Repositorio con ramas protegidas, scaffolding de <code>servidor/</code> y <code>movil/</code> , CI que corre typecheck y pruebas, development build de Expo, RevenueCat en sandbox	La app abre en un teléfono real y <code>Purchases.configure()</code> no lanza error

3.3 Semana 2 — Flujo del docente (16 al 22 de agosto)

Objetivo: un docente crea su curso y registra su primera acción.

Rol	Tareas	Éxito de la fase
A	D1–D4 (registro, crear curso, parciales), D6 panel, D10 invitación	Un docente nuevo llega desde cero hasta tener un código de invitación
B	D11 asignar acción con buscador, validación de topes y período	Asignar -1 a un estudiante actualiza su puntaje de 60 a 59 en pantalla
C	P1 registro del representante, P11 payroll, notificaciones push	Llega una notificación push a un teléfono real

3.4 Semana 3 — Flujo del representante (23 al 29 de agosto)

Objetivo: el circuito completo funciona de punta a punta.

Rol	Tareas	Éxito de la fase
A	P2 canjear código, P3 registrar hijo con consentimiento, D7 lista, D8 aprobación	Un representante registra a su hijo y el docente lo aprueba
B	D13 reporte general, P4 reporte diario del representante	El docente publica el reporte y el representante lo ve en su teléfono
C	D19 payroll del docente, límites de plan verificados en servidor	Un docente con <code>DOC_FREE</code> recibe 402 al crear su segundo curso

Hito crítico del sábado 29 de agosto: el circuito completo debe funcionar. Docente crea curso → invita → representante registra hijo → docente aprueba → docente asigna acción → docente publica reporte → representante lo ve. Si esto no está ese día, se activa el recorte de la semana 4.

3.5 Semana 4 — Punto de control (30 de agosto al 5 de septiembre)

Objetivo: cerrar todo lo obligatorio y decidir el recorte.

Rol	Tareas	Éxito de la fase
A	Selector de hijos, importación CSV, pruebas S-1 a S-7	Las siete pruebas de seguridad pasan
B	P5 reportes anteriores, P6 acumulado con gráfico	El gráfico de evolución se dibuja con datos reales
C	D17 alertas con reautenticación, P10 confirmación	Una alerta llega a los representantes del curso

Decisión del miércoles 2 de septiembre. Si el bloque obligatorio no está cerrado, se sacrifica **todo** lo opcional sin discusión y los tres pasan a pulir. Es preferible entregar cinco funciones sólidas que ocho a medias.

3.6 Semana 5 — Funciones de apoyo (6 al 12 de septiembre)

Rol	Tareas	Éxito de la fase
A	Guías de uso, ajustes del flujo de vinculación	Alguien externo usa la app sin que le expliquen
B	D12 asistencia, D14 comunicados y eventos	Un evento aparece y desaparece en las fechas correctas
C	D15 inconformidades, D16 citas, P7, P8	Un representante reclama y el docente resuelve, devolviendo puntos

3.7 Semana 6 — Cierre (13 al 19 de septiembre)

Los tres trabajan en lo mismo:

- Datos de demostración creíbles: un curso con 25 estudiantes ficticios, dos semanas de historial, puntajes variados que muestren las franjas.
- Corrección conjunta de errores.

- Video de demostración: el circuito completo en **menos de 2 minutos** (regla de Next Gen), mostrando RevenueCat en funcionamiento, con descripción y subtítulos en inglés.
- README verificado en una máquina limpia por alguien que no lo escribió.
- Repositorio hecho público con `LICENSE` visible.
- Envío el **28 de septiembre**. No dejar el envío para el último día.

Parte 4 — Mapa de documentos

4.1 Qué consultar según tu duda

Tu duda	Consulta
¿Qué hace exactamente esta función?	<code>docs/00-producto/</code>
¿Por qué elegimos esta tecnología?	<code>docs/01-arquitectura/adr/</code>
¿Quién puede ver este dato?	<code>docs/01-arquitectura/matriz-permisos.md</code>
¿Qué devuelve este endpoint?	<code>api/openapi.yaml</code>
¿Cómo se llama este campo?	<code>db/schema/</code>
¿Qué valores admite este estado?	<code>db/schema/enums.ts</code>
¿Qué me toca esta semana?	<code>docs/02-equipo/backlog-y-reparto.md</code> y tu manual de rol
¿Cómo hago un PR?	<code>docs/02-equipo/manual-equipo.md</code>
¿Qué decidimos sobre X regla?	<code>Contexto/DECISIONS.md</code>
¿Qué está pendiente de definir?	<code>Contexto/CONTEXT.md</code> , sección 7
¿Qué corrigió este manual respecto a agosto?	<code>docs/99-archivo/ERRATA-2026-08.md</code>

4.2 Estructura del repositorio

```
cresco/
├── Contexto/          <- CLAUDE.md, CONTEXT.md, DECISIONS.md, NEXT_STEPS.md
├── README.md · LICENSE
├── .env.example
├── package.json · package-lock.json
├── api/openapi.yaml   <- contrato de API
├── db/
│   ├── schema/       <- esquema Drizzle (4 archivos: enums + 1 por persona)
│   ├── migrations/   <- generadas, NUNCA se editan a mano
│   ├── seeds/        <- datos de catálogo y ficticios
│   └── acceso/        <- capa de acceso (Persona A)
└── servidor/         <- Next.js. La scaffoldea C el día 1 (npx create-next-app)
```

```

|   └─ lib/puntaje.ts <- de Persona B
|─ movil/                <- Expo, UNA app con enrutado por rol. La scaffoldea C
|   |                    el mismo día (npx create-expo-app)
|   └─ theme/Theme.ts <- de Persona A
|   └─ components/base/ <- de Persona A
└─ docs/
   └─ README.md        <- índice; empieza aquí
   └─ 00-producto/
   └─ 01-arquitectura/
   └─ 02-equipo/
   └─ 03-piloto/
   └─ 04-guias/
   └─ 05-validacion/

```

Por qué no hay un envoltorio `apps/` ni un `packages/` compartido: `servidor/` y `movil/` tienen cada uno su propio `package.json`, sin `npm workspaces`. Revisando el flujo real de datos, no hay código que de verdad se comparta entre las dos apps más allá de `db/schema/` (que ya está en la raíz y solo la importa `servidor/`): el servidor calcula todo y el móvil solo renderiza lo que la API devuelve. Esto no reabre ADR-002 ("sin monorepo completo") — sigue sin hacer falta ninguna herramienta de workspaces. **No crees `servidor/` ni `movil/` a mano ni por adelantado:** las herramientas de scaffolding (`create-next-app`, `create-expo-app`) suelen rechazar un directorio destino que ya tiene contenido.

Parte 5 — Riesgos

Riesgo	Impacto	Mitigación	Dueño
RevenueCat no funciona en dispositivo real	Fatal: incumple el Shipaton	Probarlo en semana 1, no al final	C
A se atrasa y bloquea a B y C	Alto	Servidor simulado desde el día 1	Todos
Alcance crece sin control	Alto	Punto de control del 2 de septiembre	Todos
Alguien se cae por exámenes	Medio	Avisar temprano; reasignar en el punto de control	Todos
Conflictos de fusión constantes	Medio	Regla de oro: nadie toca módulo ajeno	Todos
Fuga de datos de un menor	Fatal para el producto	Dos capas de permisos + pruebas S-1 a S-7	A
Repositorio público sin LICENSE a tiempo	Alto — bloquea el envío	Elegir licencia esta semana; agregar LICENSE antes de semana 6	C

El trámite de Google Play Console **ya no es un riesgo del proyecto**: las reglas de Next Gen no lo exigen (verificado el 7-8 de agosto). Ver [Contexto/reglas-shipaton-next-gen.md](#).

Parte 6 — Bloque de contexto para IA

Cómo usarlo: copien todo lo que sigue y péguenlo antes de su pregunta cuando le pregunten a una inteligencia artificial. Así la respuesta llega ajustada a este proyecto en vez de ser genérica.

CONTEXTO DEL PROYECTO – CRESCO (equipo Neofix)

Somos 3 estudiantes de ingeniería en computación construyendo una app Android para el hackathon RevenueCat Shipaton 2026 (categoría Next Gen). Semana 1 desde el 9 de agosto de 2026. Deadline interno 20 de septiembre; envío real 28 de septiembre; cierre oficial del Shipaton 30 de septiembre 23:45 PDT. Dos de los tres nunca hemos construido una app antes.

QUÉ ES: app de comunicación entre docentes y representantes legales de escuelas del Ecuador, para seguimiento de conducta y responsabilidad. El estudiante NUNCA es usuario. Dos interfaces: docente (herramienta de trabajo) y representante (informativa, simple, para personas con poca alfabetización estadística).

NEXT GEN NO EXIGE cuenta de desarrollador de Google Play ni publicación en tienda: se evalúa por video de demostración (<2 min) y repositorio público de código abierto con LICENSE en la raíz.

STACK (decidido, no se rediscute):

- PostgreSQL + Drizzle ORM
- Next.js como SERVIDOR (no frontend web): BetterAuth, rutas API, webhook de RevenueCat, tarea nocturna de reportes
- Expo / React Native: UNA sola app, con enrutado por rol (docente/representante)
- BetterAuth con plugin de organización (organization = institución)
- RevenueCat SDK para suscripciones (REQUISITO OBLIGATORIO del hackathon)
- NativeWind para estilos. shadcn/ui NO se usa en las apps: es solo web.

REGLAS DE NEGOCIO CLAVE:

- Puntaje: base 60 por parcial, piso 0, techo 100. Se reinicia cada parcial. Es DERIVADO: se recalcula sumando acciones VIGENTES, nunca es un contador que se incrementa.
- Positivas: +1 por defecto, ajustable a +2.
- Negativas de responsabilidad: fijo -1. De disciplina: -1 a -3.
- Topes diarios: máximo +4 y máximo -5 por estudiante por día.
- Acciones anuladas o modificadas valen 0 pero se muestran distinto.
- Solo las acciones negativas se pueden anular y reclamar.
- 5 tipos de acción: Positiva, Negativa, Nota (por estudiante) + Anuncio,

- Evento (por curso, en tabla separada comunicado_curso).
- Un solo representante legal por estudiante en la v1.
- El representante registra los datos de su hijo; el docente los aprueba.
- La invitación es al CURSO, no a un estudiante, y dura 30 días.
- No hay administrador de institución en la v1; el docente escribe el nombre de su escuela como texto libre.
- No hay rol de docente colaborador en v1 (diferido a v2), aunque el campo exista en el esquema.
- Reportes: el docente llena campos etiquetados, ninguno obligatorio. Solo el más reciente es visible en la pantalla principal del representante. Plan gratuito: 2 reportes anteriores. Premium: 7.
- Alerta de emergencia: exige que el docente reingrese su contraseña.
- Citas en bloques de 15 minutos, requieren confirmación del docente.

SEGURIDAD (crítico): un representante solo accede a estudiantes con vínculo ACTIVO. Se aplica en DOS capas: capa obligatoria de acceso a datos en el servidor y políticas RLS de PostgreSQL. Cada petición abre transacción con SET LOCAL app.user_id.

ESTRUCTURA DEL EQUIPO (Neofix):

- Persona A: identidad, estructura académica, vinculación, permisos
- Persona B: acciones, puntaje, asistencia, reportes
- Persona C: citas, alertas, RevenueCat, notificaciones, infraestructura

CONVENCIONES: nombres de tablas y columnas en español, snake_case. Estados en MAYUSCULA_SNAKE definidos en db/schema/enums.ts. Timestamps timestamptz en UTC, zona America/Guayaquil – pero las columnas `date` (p. ej. fecha_ingreso) las calcula el servidor explícitamente, nunca `defaultNow()`, porque `now()::date` puede caer en el día equivocado según el timezone de la sesión de la base de datos. Nada se borra físicamente; se anula o se cambia de estado.

ESTRUCTURA DE CÓDIGO: servidor/ (Next.js) y movil/ (Expo) son carpetas hermanas en la raíz del repo, cada una con su propio package.json, SIN npm workspaces – no es un monorepo. servidor/lib/puntaje.ts es de B; movil/theme/ y movil/components/base/ son de A. Persona C las scaffoldea el día 1.

RESTRICCIONES: 6 semanas, 3 personas, dos sin experiencia previa. Preferimos soluciones simples y probadas sobre soluciones elegantes.

Versión 1.2 · Última revisión: 8 de agosto de 2026 · Dueño: el equipo · Corrige la v1.0 del 3 de agosto — detalle en <docs/99-archivo/ERRATA-2026-08.md>